

Ödev 03: Notlar ve Izgaralar

İki küçük program. Birincisi işe uygun koşul yapısını seçmenizi istiyor. İkincisi, etiketli break'in neden var olduğunu önce onsuz yaşatarak gösteriyor.

Birinci Bölüm: Notlar

Grades.java oluşturun. Şu puan listesi verildiğinde:

```
int[] scores = {95, 83, 71, 64, 42, 100, 0};
```

Her puanı harf notu ve kısa bir yorumla yazdırın, sonra sınıf ortalamasını notuyla birlikte yazdırın.

Not sınırları: 90 ve üzeri A, 80 ve üzeri B, 70 ve üzeri C, 60 ve üzeri D, altı F.

Hedef çıktı:

```
95 A excellent
83 B solid
71 C passing
64 D scraped through
42 F see me
100 A excellent
0 F see me
```

```
Class average: 65 (D)
```

Vermeniz Gereken Karar

Bu iki işten biri switch ifadesine uygun, diğeri değil.

- Bir **puanı** harfe çevirmek
- Bir **harfi** yoruma çevirmek

Hangisinin hangisi olduğunu bulun, switch yalnızca uyduğu yerde kullanın ve diğerinin neden if zinciri olduğunu bir yorumda açıklayın. Uymadığı yere switch zorlamak, hiç kullanmamaktan daha kötü bir cevaptır.

İkinci Bölüm: Izgarada Arama

FindInGrid.java oluşturun. Bu ızgarada bir hedef değer arayın ve satır ile sütununu bildirin, ya da bulunmadığını söyleyin:

```
int[][] grid = {
    { 7, 12, 3},
    { 9, 5, 21},
    {14, 2, 18}
};
```

Aramayı **iki kez** yazın:

1. `withFlag` — bir `boolean found` değişkeni ve düz `break` kullanarak
2. `withLabel` — etiketli break kullanarak

İkisini de mevcut olan `21` ve olmayan `99` için çalıştırın.

Sonra ikisini karşılaştıran bir yorum yazın. Özellikle: bayraklı sürümün gerektirdiği ve ızgarada aramayla hiç ilgisi olmayan şeyleri sıralayın.

Kabul Kriterleri

- ☐ Her iki dosya da `java Grades.java` ve `java FindInGrid.java` ile çalışıyor
- ☐ `Grades.java` iki işten tam olarak biri için `switch` ifadesi kullanıyor
- ☐ Bir yorum, diğerinin neden switch olmadığını açıklıyor
- ☐ Sütunlar hizalı; `100` ve `0` ikisi de doğru görünüyor
- ☐ `FindInGrid.java` hem `withFlag` hem `withLabel` içeriyor
- ☐ Her iki sürüm de `21` ve `99` için birebir aynı çıktıyı üretiyor
- ☐ Bir yorum, bayraklı sürümün gerektirdiği fazladan işleri sıralıyor

İleri Seviye

Her iki döngüden de çıkmanın, ne bayrak ne etiket gerektiren üçüncü bir yolu var. Bulun, `withExtractedMethod` olarak uygulayın ve etiketli break'e ne zaman tercih edeceğinizi söyleyin.

İpucu, switch derlenmiyorsa

Bir switch ifadesi her olası girdi için bir değer üretmek zorundadır. `int` ya da `String` üzerinde bu, durumları sayarak her şeyi kapsayamayacağınız anlamına gelir, dolayısıyla derleyici `default` ister. Her sabiti listelenmiş bir `enum` üzerinde ise istemez. Bu fark, iki işten birinin neden switch'e daha uygun olduğunun tüm sebebi.